Jenkins Pipeline for SeleniumApp1 (Auto-sausedemo)

Step 1: Open the Jenkins Dashboard in order to create a new item.

Step 2: Click on New Item in order to create a New Item. Give the name as "SeleniumApp1" and Item type as Pipeline, then click Create.

Step 3: Configure the settings by defining the Definition as "Pipeline Script from SCM" and SCM as "Git". Paste the Repository URL of the project from GitHub and choose Credentials as "GitHub Credentials" which was created earlier.

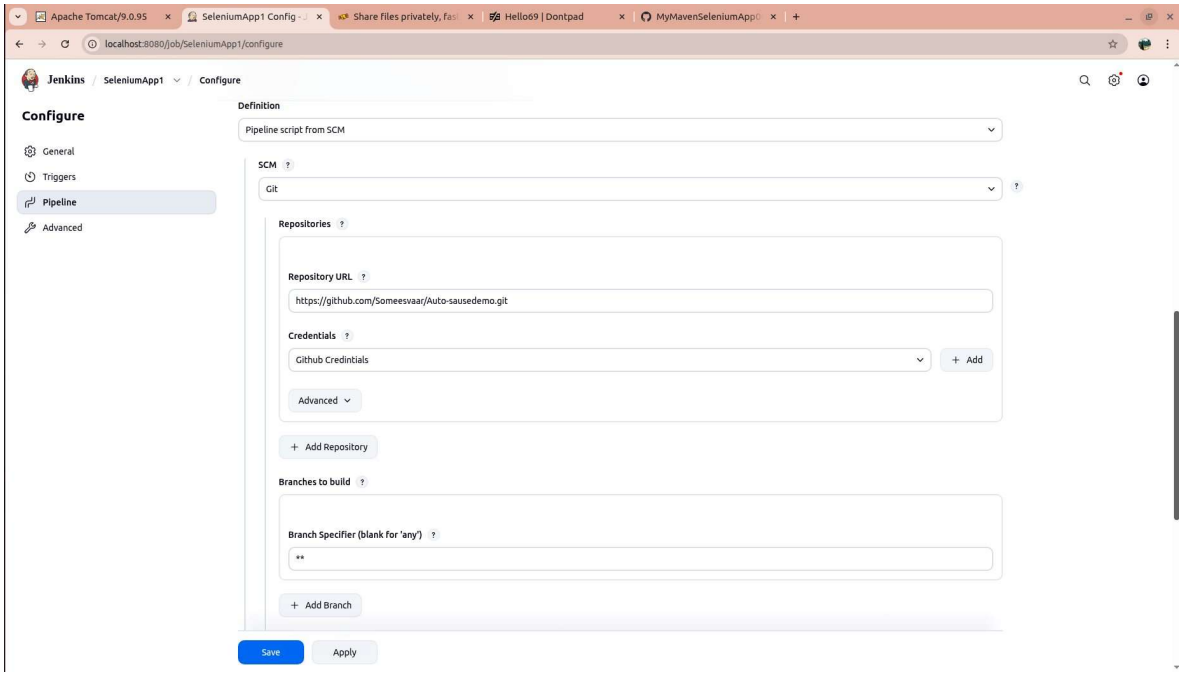

Fig A-1: Jenkins Pipeline configuration – SCM set to Git with Repository URL for Auto-sausedemo.git and GitHub Credentials

Step 4: In Terminal, go to the directory created for SeleniumApp1 and create a new file called "Jenkinsfile".

Step 5: Paste the following code inside the Jenkinsfile:

```
pipeline {
    agent any  // Use any available agent

    tools {
        maven 'Maven'  // Ensure this matches the name configured in Jenkins
        jdk 'JDK'
    }
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main', url: 'https://github.com/Someesvaar/Auto-sausedemo.git'
            }
        }

        stage('Build') {
```

```
            steps {
                sh 'mvn clean package'  // Run Maven build
            }
        }

        stage('Test') {
            steps {2
                sh 'mvn test'  // Run unit tests
            }
        }




        stage('Run Application') {
            steps {
                // Start the JAR application
                sh 'java -jar target/MyMavenSeleniumApp01-1.0-SNAPSHOT.jar'
            }
        }



    }
    post {
        always {
            echo 'Pipeline completed.'
        }
    }
}
```

And add the following to App.java
options.addArguments("--headless=new"); // modern headless mode
options.addArguments("--no-sandbox"); // required in Jenkins/Linux
options.addArguments("--disable-dev-shm-usage"); // prevents crashes
options.addArguments("--disable-gpu");

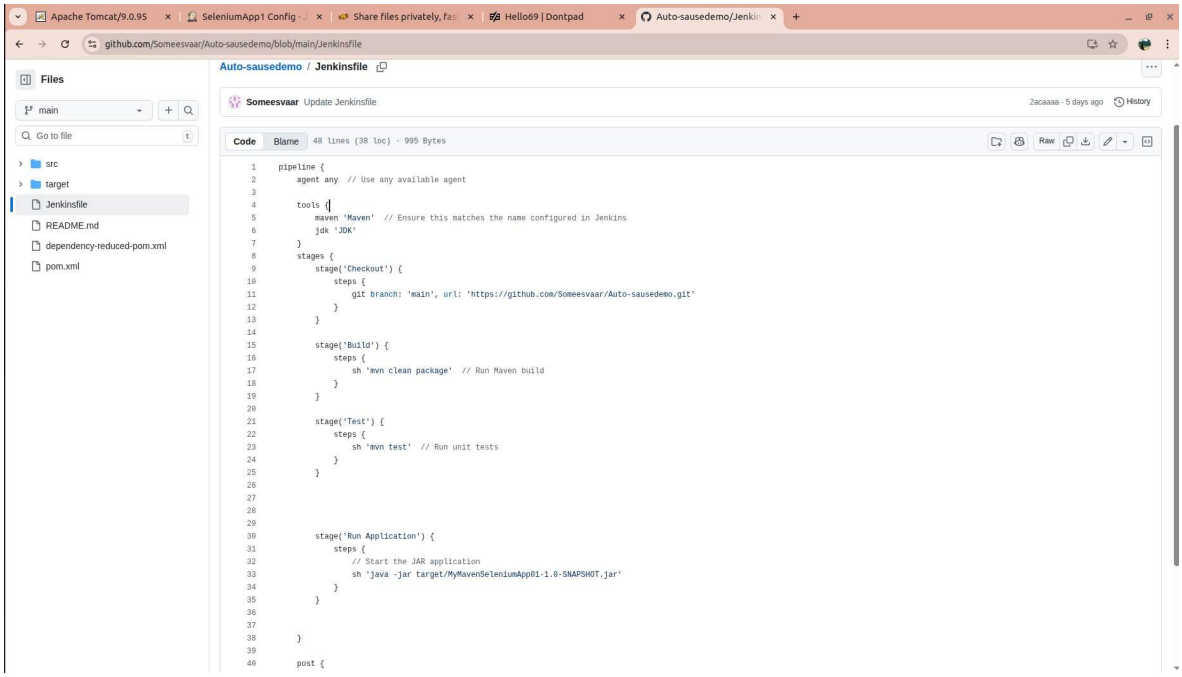

Fig A-2: Jenkinsfile committed to the Auto-sausedemo GitHub repository

Step 6: Update pom.xml to change the versions of source and target from 1.7 to 1.8.

Step 7: Push the changes and commit by pushing the "Jenkinsfile" to the respective repository on GitHub.

Step 8: Go to Jenkins and Build the Pipeline, then check the Stage View to confirm successful execution.

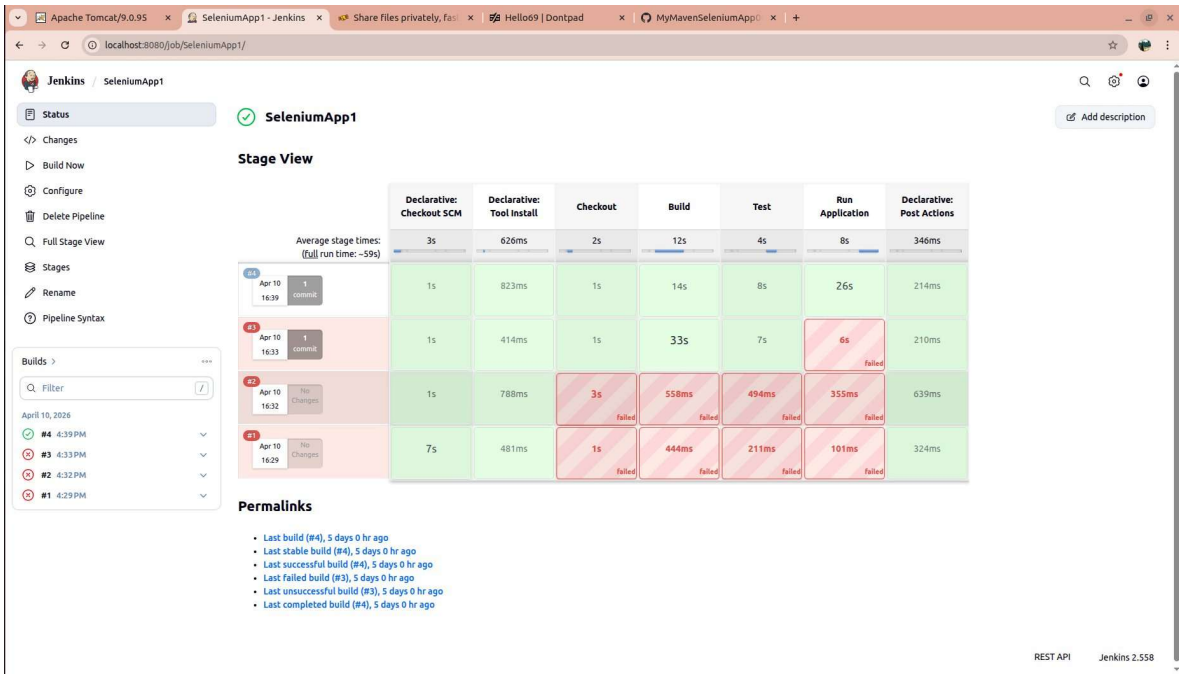

Fig A-3: Jenkins Stage View showing Build #4 successful for SeleniumApp1 (~59s total)